

Professores:

Celso Rodrigo Giusti (The Best)

Daniel Manoel Filho

Felipe Augusto Stevanim Gregate

Marlon Palata Fanger Rodrigues

REACT • AULA 2

Estrutura do Projeto e Interatividade



O que faremos hoje

- ✅ **Aula passada:** criamos o projeto com Vite, componentes, props e `.map()`.
- **Parte 1:** a anatomia do projeto — cada pasta e cada arquivo.
- **Parte 2:** eventos (`onClick`) e `useState` — a tela passa a reagir.
- 🎯 **Fio condutor:** o TechFood ganha botões que funcionam de verdade.

Anatomia do projeto — visão geral

techfood-react/

- |— node_modules/ ← bibliotecas instaladas
- |— public/ ← arquivos públicos (ícones)
- |— **src/** ← **o SEU código mora aqui**
- |— index.html ← só tem o <div id="root">
- |— package.json ← o RG do projeto
- |— package-lock.json ← versões exatas
- |— vite.config.js ← configuração do Vite
- |— eslint.config.js ← regras de boas práticas
- |— .gitignore · README.md

index.html e main.jsx

index.html: a ÚNICA página do app (é uma SPA).

- `<div id="root"></div>` ← o React desenha aqui dentro
- `<script type="module" src="/src/main.jsx">`

main.jsx: o arquivo que LIGA o React na página.

- `createRoot(document.getElementById("root"))`
- `.render(<App />)`

⚠ Quase nunca mexemos neles — mas precisamos entender.

package.json e package-lock.json

- **package.json** = o RG do projeto: nome, scripts e dependências.

- "dev": "vite" → é isso que roda no npm run dev

- "react": "^19.x" ← ^ aceita atualizações menores

- **package-lock.json** = a versão EXATA de cada pacote.

Garante que todos instalem exatamente igual.

- ⚠ Nunca edite o package-lock.json na mão.

node_modules e .gitignore

- **node_modules/** = onde o npm guarda as bibliotecas.

Pesada (milhares de arquivos) e recriável.

- npm install → recria tudo a partir do package.json

- **.gitignore** = a lista do que o Git NÃO envia.

- node_modules · dist · .env

- 💡 Clonou um projeto? Primeiro comando: **npm install**.

vite.config.js, eslint.config.js e README

- **vite.config.js** — configura o Vite (plugins, porta...).
O plugin do React já vem ligado.
- **eslint.config.js** — regras de boas práticas.
O ESLint sublinha problemas enquanto você digita.
- **README.md** — a "capa" do projeto no GitHub.
Como rodar, o que é e quem fez.
- ⚠ Mexemos pouco: configuração pronta = produtividade.

public/ × src/assets/

- **public/** — arquivos servidos do jeito que estão.
 - /vite.svg → acessado direto pela URL

- **src/assets/** — imagens usadas DENTRO dos componentes.
 - import logo from "./assets/logo.png"
 -

- 💡 **Regra prática:** imagem do componente → assets. Ícone da aba → public.

src/ — onde o seu código mora

- **main.jsx** — liga o React no #root.
- **App.jsx** — componente raiz: monta a tela.
- **App.css** — estilos do App.
- **index.css** — estilos globais (body, fontes).
- **components/** — um componente por arquivo.
- **data/** — dados separados da tela (pasta nova de hoje).

💡 Organização = achar as coisas rápido.

Componente — a peça da tela

Componente = uma função JavaScript que devolve um pedaço da tela (JSX).

```

● function Header() {
●   return <header><h1>TechFood</h1></header>;
● }
● export default Header;
  
```

✓ Nome em **Maiúscula** (PascalCase) · retorna **UM** elemento pai.

✓ Usa como tag: **<Header />** — quantas vezes quiser.

Props — dados para o componente

Props = os "parâmetros" do componente. O PAI passa, o FILHO recebe.

- `<CardPrato nome="Feijoada" preco={42.90} />`
- `function CardPrato({ nome, preco }) {`
- `return <h2>{nome}</h2>;`
- `}`

✓ Texto → aspas · número/JS → chaves { }

⚠ Props são só leitura: o filho usa, mas não altera.

Listas com `.map()` e `key`

.map() percorre um array e devolve UM componente para cada item.

```
{cardapio.map((prato) => (  
  <CardPrato key={prato.id} nome={prato.nome} />  
))}
```

- ✓ key = identidade única de cada item (use o **id**).
- ⚠ Nunca use o índice como key. Sem key → aviso no console.
- 💡 1 array + 1 `.map()` = N cards, sem copiar e colar.

Juntando tudo



1. O **App** tem os dados (o array cardapio).
2. O **.map()** cria um CardPrato para cada prato.
3. Cada CardPrato recebe os seus dados por **props**.
4. E desenha o seu próprio card.

 **Os dados DESCEM: do pai para o filho, sempre de cima pra baixo.**

 É exatamente o que vocês montaram na aula passada.

import e export

Export default: o principal do arquivo (só 1).

- export default Header;
- import Header from "./components/Header";

Export nomeado: vários por arquivo, com { }.

- export const cardapio = [...];
- import { cardapio } from "./data/cardapio";

⚠ Nomeado usa chaves e o MESMO nome do export.

O fluxo completo

1. O navegador abre o **index.html**.
2. O index.html carrega o **main.jsx**.
3. O main.jsx desenha o **«App /»** dentro do #root.
4. O App.jsx importa e monta os **componentes**.
5. Cada componente devolve o seu pedaço de JSX.

 **index.html** → **main.jsx** → **App.jsx** → **componentes**

Eventos — onClick

Evento = algo que o usuário FAZ (clicar, digitar, passar o mouse).

- `function adicionar() { console.log("Adicionou!"); }`
- `<button onClick={adicionar}>Adicionar</button>`

- ✓ `onClick` com C maiúsculo (camelCase).
- ✓ Passe a função SEM parênteses: `{adicionar}`
- ⚠ `{adicionar()}` executa na hora, sem ninguém clicar!

useState — a memória do componente

Estado = um valor que, quando muda, faz a tela atualizar sozinha.

- `import { useState } from "react";`
- `const [quantidade, setQuantidade] = useState(1);`
- `<button onClick={() => setQuantidade(quantidade + 1)}>+</button>`

✓ `quantidade` = valor atual · `setQuantidade` = muda o valor

⚠ Nunca faça `quantidade = 5`. Use SEMPRE o `set`.

O que vamos fazer:

1. Tour pela estrutura (arquivo por arquivo).
2. Separar o cardápio em data/cardapio.js (export nomeado).
3. Botão "Adicionar" com onClick.
4. Quantidade - / + em cada card com useState.
5. Bônus: contador do pedido no Header.

Ver **Guia de Live Coding** para o passo a passo completo.

Exercícios — TechFood interativo

Parte A — Estrutura: responda as 3 perguntas do EXERCICIOS.md.

- **NÍVEL 1:** novo prato em data/cardapio.js · botão ❤️ Curtir com contador.
- **NÍVEL 2:** limite de 10 na quantidade · mostrar/esconder descrição.
- **DESAFIO:** botão "Limpar pedido" · total do pedido em R\$.

📁 **Entrega:** print + PDF no Classroom (AULA04_SEUNOME.pdf).

Resumo



- ✓ src/ é o seu código; o resto é configuração e ferramenta.
- ✓ package.json = RG · package-lock.json = versões exatas.
- ✓ node_modules não vai pro Git — npm install recria.
- ✓ import/export conectam os arquivos (default × nomeado).
- ✓ Eventos: onClick={funcao} — sem parênteses.
- ✓ useState: valor + set. Mudou o estado, a tela atualiza.

A seguir: **aprofundar o estado — formulários e listas.**

Referências

REACT TEAM. React Documentation — Adding Interactivity. Disponível em: react.dev/learn. Acesso em: set. 2026.

VITE TEAM. Vite — Guide. Disponível em: vitejs.dev/guide. Acesso em: set. 2026.

NPM. package.json e package-lock.json. Disponível em: docs.npmjs.com. Acesso em: set. 2026.

MOZILLA. MDN Web Docs — Módulos JavaScript. Disponível em: developer.mozilla.org/pt-BR. Acesso em: set. 2026.



FOTO DA ESCOLA

ESCOLA SENAI ÍTALO BOLOGNA

Av. Goiás, 139

Telefone

(11) 2396-1999

Instagram

@senai.itu

Facebook

/senaisp.itu

Site

<https://sp.senai.br/unidade/itu/>